

Matrix chain Multiplication

$$M[i,j] = \min \{ m[i,k] + m[k,j] + P_{i-1}P_kP_j \}$$

Matrix(P)

$$n = \text{length}(P) - 1$$

for $i = 1$ to n

$$M[i,i] = 0$$

for $l = 2$ to n

for $i = 1$ to $n - l + 1$

$$j = i + l - 1$$

$$M[i,j] = \infty$$

for $k = i$ to $j - 1$

$$q = m[i,k] + m[k+1,j] + P_{i-1}P_kP_j$$

if $q < m[i,j]$

$$m[i,j] = q$$

$$s[i,j] = k$$

$O(n^3)$

0/1 Knapsack

$$\text{Max} \{ \text{DP}(w, k), \text{DP}(k+1, w - w_k) + \text{val}[k] \}$$

Knapsack(k, w)

if $k = 0$ (or) $w = 0$

return 0

if $s[k, w] \neq -1$

return $s[k, w]$

if $(wt[k] \leq w)$

$$s[k, w] = \max \{ \text{val}[k] + \text{knapsack}(k-1, w - wt[k]), \text{knapsack}(k-1, w) \}$$

return $s[k, w]$

else

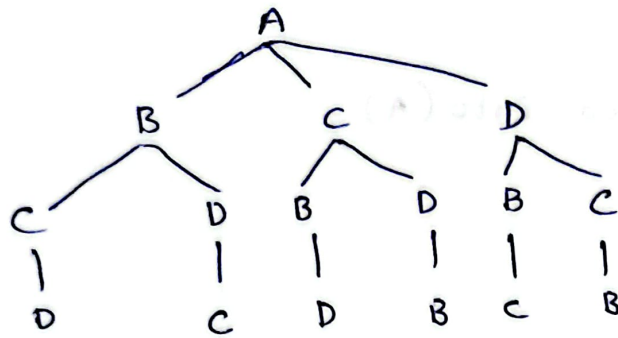
$$s[k, w] = \text{knapsack}(k-1, w)$$

return $s[k, w]$

$(O(nw))$

P	W	0	10	20	30	40	50
0	0	0	0	0	0	0	0
100	10	0	100	100	100	100	100
100	20	0	100	100	200	200	200
120	30	0	100	100	120	220	220

Travelling Span Salesman problem



$$TSP(i, S) = \min_{k \in S} \{ C_{ik} + g(k, \{S\} - \{k\}) \}$$

$$g(1, \{2, 3, 4\}) = \min_{k \in \{2, 3, 4\}} \{ C_{1k} + g(k, \{2, 3, 4\} - \{k\}) \}$$

TSP(S, k, L)

if (S == 1111)

print(L)

return W[k, 1]

cost = ∞.

for j = 1 to n

if ((1 < n - j) && S == 0)

return

W[k, j] + TSP(S | 1 < n - j, j, L.append[j])

if (C < cost)

cost = C

m[S, k] = cost

return cost

• S is a bit mask

k current city

L is the path

W[k, j] is cost of path

$O(n^2 2^n)$

All pair Shortest path

APSP(d, n)

for k = 1 to n

for ~~j~~ i = 1 to n

for j = 1 to n

if (k ≠ i & k ≠ j)

$$d[i, j] = \min \{ d[i, j], d[i, k] + d[k, j] \}$$

$$d[i, j] = \min \{ d[i, j], d[i, k] + d[k, j] \}$$

Here if k ≠ i

$O(n^3)$

* Greedy Algorithm

Greedy (A)

$A = \text{candidate}(a) / A = \text{ordered-into}(A)$

$Sol = \emptyset$

for $i=1$ to n

$x = \text{select}(A[i])$

if feasible (Sol, x):

$Sol = Sol + x$

return Sol

* Job-sequencing with deadline

JSD (J, D, n)

$J = \text{Job-sort}()$

$T = []$

flag = 0

for $i=1$ to n :

for $k = D[i]$ down to 1:

if $T[k] = \emptyset$:

$T[k] = J[i]$

flag = 1

break

if flag = 0 $\rightarrow$ break / if (flag $\geq$ Max(D)):

break

return T .

* Kruskal's Algorithm (MST):

Kruskal (V, E):

for each $u \in V$:

make set (u)

$E = \text{sort}(E, \text{non-decreasing order of weight})$

$T = \emptyset$

for each $(u, v) \in E$:

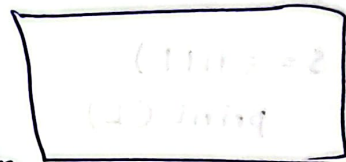
if (findset (u) $\neq$ findset (v)):

union (findset (u), findset (v))

$T = T \cup \{(u, v)\}$

return T

$k \rightarrow D[i]$



(J, D, n)

$J = \text{Job-sort}$

$T = []$

for $i=1$ to n

for $k = D[i]$ down to 1

if $T[k] = \emptyset$

$T[k] = J[i]$

flag = 1

break

prim's Algorithm

prim(v, E):

$Q = G(v)$

for $\forall u \in Q$:

key[u] = ∞

P[u] = NULL

key[a] = 0

P[a] = a

while $Q \neq \emptyset$:

$u = \text{ExtractMin}(Q)$

for $\forall v \in \text{Adj}(u)$:

if key[v] > $w[u, v]$:

key[v] = $w[u, v]$

P[v] = u

DecreaseKey($Q, v, \text{key}[v]$)

Dijkstra(v, E):

Dijkstra(v, E):

$Q = G(v)$

for $\forall u \in Q$:

key[u] = ∞

P[u] = NULL

key[a] = 0

P[a] = a

while $Q \neq \emptyset$:

$u = \text{ExtractMin}(Q)$

for $\forall v \in \text{Adj}(u)$:

if key[v] > $d[u] + w[u, v]$:

$d[v] = d[u] + w[u, v]$

P[v] = u

DecreaseKey($Q, v, d[v]$)

key[a] = 0

P[a] = a

Elog V while $Q \neq \emptyset$:

for $\forall v \in \text{Adj}(u)$:

$u = \text{ExtractMin}(Q)$

if key[v] > $w[u, v]$

key[v] =

key[u]

+ $w[u, v]$

P[v] = u

Disjoint set :

MakeSet(i):

return i

FindSet(a):

return Id[a]

union(a₁, a₂)

Id1 = FindSet(a₁)

Id2 = FindSet(a₂)

if (Id1 ≠ Id2):

for v=1 to n:

if Id[v] = Id2:

Id[v] = Id1

Ex: union(1, 2)

union(a₁=1, a₂=2)

Id1 = FindSet(a₁) → 1

Id2 = FindSet(a₂) → 2

if Id1 ≠ Id2:

for v=1 to n:

~~if Id[v] = Id1~~

if Id[v] = Id2 → ②

Id[v] = Id1 → (1)

union(1, 2) {1} {2}

for v=1 to n

if Id[v] = 2

Id[v] = 1

Ex:

Start: {1} {2} {3}

union(1, 2) → {1, 2} {3}

FindSet(2) = FindSet(1)

Id[1] = 1

Id[2] = 2

Id[3] = 3

Id[4] = 4

{1} {2} {3} {4}

Id[1] = 1

Id[2] = 1

Id[3] = 3

Id[4] = 4

} union(1, 2)

TSP problem

Dynamic program $\rightarrow$ To find optimal solutions $\rightarrow O(n^2 2^n)$

Back Tracking $\rightarrow$ To find all possible solutions $\rightarrow O(n!)$

Branch & Bond $\rightarrow$ To find optimal solutions $\rightarrow O(2^n)$

TSP_BKT (visited, cur-Ind, Count, cost)

Global G, n, Bound, Start Ind

if (cost + G[cur-Ind][Start-Ind] > Bound)

return

if count == n

if (cost + G[cur-Ind][Start-Ind] < Bound)

Bound = cost + G[cur-Ind][Start-Ind]

print (visited, cost)

return

for i = 1 to n

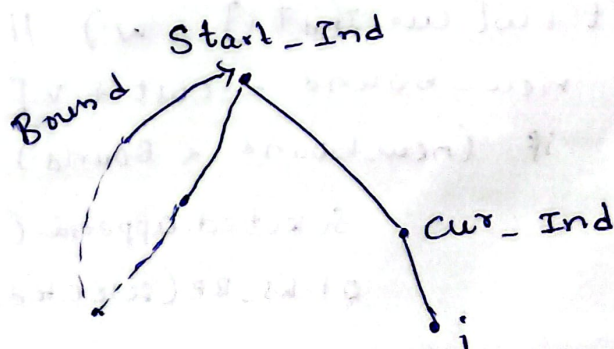
if feasible (visited, i, count)

~~visited~~
if (new_bound < Bound)

visited[count] = i

TSP_BKT (visited, i, count+1, G[cur-Ind][i])

visited[count] = -1.

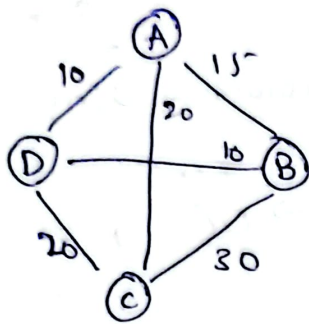


$\text{cost} + G[\text{cur-Ind}][i] + G[i][\text{Start-Ind}]$

new_bound

* Recursion is always maintained by stacks.

And Stacks always use Depth First Traversal (DFS)



$$LC = \begin{array}{|c|c|c|c|} \hline A & B & C & D \\ \hline 10 & 10 & 20 & 10 \\ \hline \end{array}$$

0/1 knapsack To get minimum using Bounds.

$W = 60$, $n = 4$, $Bound = \infty$.

$$W = \begin{array}{|c|c|c|c|} \hline 10 & 20 & 30 & 20 \\ \hline \end{array}$$

$$V = \begin{array}{|c|c|c|c|} \hline 100 & 140 & 150 & 160 \\ \hline \end{array}$$

$$P = V/W \quad \begin{array}{|c|c|c|c|} \hline 10 & 7 & 6 & 8 \\ \hline \end{array}$$

Sorted the above $\uparrow$

0/1_KS_BB (selected, cur-Ind, cost, wt)

Global w, v, p, n , Bound

if (cost > Bound)

return

} on hold

if (cur-Ind == n || wt == w)

if cost < Bound

Bound = cost

print (selected, cost)

if (wt + w[cur-Ind+1] < w) // Feasibility

new-bound = cost + v[cur-Ind+1]

if (new-bound < Bound)

selected.append (cur-Ind+1)

0/1_KS_BB(selected, cur-Ind+1, cost +

v[cur-Ind+1],

wt + w[cur-Ind+1])

selected.pop()